

Homework 4

GWU CSCI 1112 - Spring 2026

March 5, 2026

1 Introduction

This homework is intended to reinforce your understanding of working with abstract Lists and their implementations whether array based or linked-list based. Your task is to implement both an ArrayList and a LinkedList to support a music catalog application using the provided framework.

1.1 Evaluation Date

March 20, 2026 at the beginning of your scheduled lab

1.2 Evaluation Method

Handwritten programming quiz where you will need to recount in pseudo-code aspects of this programming homework. The manual programming tasks will be relatively simple, but they will require a student to have practiced the homework to immediately understand and answer the question. Students who have not practiced or at least studied this homework, will generally be guessing during the quiz and they will typically require a long time to complete the quiz. Quizzes are timed and that time is capped at 30 minutes for all students including DSS students as this quiz involves recitation of already learned and practiced concepts and is not asking for full creative thinking. *Someone who has done this work should be able to finish the associated programming quiz in less than 10 minutes. If you have a handwriting disability, please talk to the professor before the evaluation date.*

1.3 Grading Rubric

I leave this here as an artifact for your reference. This is how this homework was graded when we evaluated student implementations. A rubric was provided with each homework and as the homeworks progressed, the percentages would encompass more critical metrics.

If your program does not compile and run without exceptions, the entire rubric is invalidated and the coding exercise will receive a zero score.

Coding Exercise:

- 20% For array list implementation
- 20% For linked list implementation
- 20% For passing all unit tests (10% for the array list and 10% for the linked list)
- 30% For sufficiency of comments (15% for ArrayList.java, and 15% for LinkedList.java)
- 10% For consistency of coding style

Analysis Exercise:

- 40% for analysis of array-list operations

- 40% for analysis of linked-list operations
- 20% for clarity of reasoning and quality of arguments

1.4 Comments

This section applies to graded student implementations. It is an artifact left for reference so you can understand expectations in the framework.

In this and future homeworks, you will have increased responsibility with commenting of code. Documentation is as critical a part of a programmer's job as programming code. In the previous frameworks, we have provided ample documentation for every file as a model for proper documentation procedures. It is your now responsibility to properly document your implementations in their entirety. This amount and type of documentation is an industry standard and we have provided sufficient examples in all of the assignments so far for you to base your work on.

In this and any future assignments, you will be expected to include the same types and detail for your documentation. There are three classes of comments:

- File header comments
- Function header comments
- Line comments

File header comments summarize the content of the file so that a programmer can quickly understand what is in the file without scanning through the file itself, provide clear natural language descriptions that can be matched by text based file search utilities, and associate authors, organizations, and copyrights with the code.

Function header comments document how to use functions in detail. The style used in this class is defined by the [Doxygen Standard](#) which facilitates the automatic generation of html web pages from code. All functions must have a comment that precedes the function declaration, *i.e.* must be before the function so that it clearly stands out, and this comment must describe 3 things when applicable: what the function does, every input into the function, and what the function returns if anything.

Line comments have a variety of roles. Typically they are used explain the internal logic within a class and often are associated with descriptions of the flow of execution inside a function. However, they are also associated with instance variables and explaining their role in the class, so they are not exclusively associated with functions. Line comments should be written in plain English and should add information that is not obvious from the code itself. Good line comments should help a reader understand the program even if that person does not understand the language or programming in general. Line comments are not necessary for every line of code, and can be used in groups to explain several steps grouped together. In general, line comments should be left justified to the same indentation as the current coding block and they should precede the upcoming code to prepare the reader for the code they are about to read.

1.5 Use of Imported Java Types

A goal of this class is to develop an understanding of how to build data structures. We will be developing our own classes for ArrayLists, LinkedLists, Stacks, Queues, BinaryTrees, etc. For this reason, you are not permitted to import any of the existing classes from any library unless explicitly stated.

2 Music Catalog

Whether a physical collection, or by utilizing an online platform, in order to organize, collect, and share music, the concept of a music catalog is used universally. This model is used by almost all online platforms; however, the details of the catalogs are implemented are opaque to the user and are probably implemented by programmers using a variety of approaches.

A music catalog is fairly easy to visualize. It is a stack of CDs next to some speakers, a library or playlist on Spotify (or alike), or a spreadsheet to manage vinyl discs in a collection. Items within the collection are instances of songs, and a user is free to transfer one or more of a song from the available song bank into their catalog. They may also choose to remove one or more songs from their catalog at any time. Once a user has decided they want to finalize and share the playlist, they publish their collection of songs where the range of years for the songs in the collection is calculated.

We will develop our own virtual music catalogs. In fact, we will develop two different catalogs, one that uses an Array based list and one that uses a linked list. Both music catalogs will share the same interface; however, you will have to implement the details differently depending on the type of data structure used. To implement the array based list music catalog, you will implement the methods in `ArrayList.java`, and to implement the linked list music catalog, you will implement the methods in `LinkedList.java`.

2.1 Framework

You will note that a number of classes have been provided as a framework: `CatalogHelper`, `Song`, `CatalogItem`, `MusicCatalog`, `UnitTests`, and `UseCases`. You will primarily focus on implementing methods in the `ArrayList` and `LinkedList` classes; however, you should familiarize yourself with the rest of the files to understand each of their roles in the project.

The `Song` class encapsulates the concept of songs that a catalog may contain which may be selected from the song bank and placed into the catalog.

The `CatalogItem` class is a wrapper class designed to make list construction a little easier and to hold properties more closely associated with the catalog than the song. For example, in this implementation the `Song` class is really a model for songs shared by everyone using the marketplace and variables like release year are not strictly a function of the song because not everyone collects the same songs/versions of songs when they were released by different bands and record companies. Note that we are not modeling full albums for this exercise, but if we did, it would appear in the `CatalogItem` class not in the `Song` class.

The `CatalogHelper` class is provided to facilitate construction of an inventory of songs for a catalog. `CatalogHelper` provides a static method, `getCatalog` which returns an array of songs.

The `MusicCatalog` class defines an abstract interface for music catalogs. A compatible music catalog must implement every method defined in this class. This is equivalent to the `List` abstraction layer we have discussed repeatedly in class.

The `ArrayList` class implements an array-list based music catalog. This is one of the classes that you must implement for this assignment. The framework clearly labels all of the methods that you must implement inside the class. You may also implement additional helper methods inside the class as you see fit.

The `LinkedList` class implements an linked-list based music catalog. This is one of the classes that you must implement for this assignment. The framework clearly labels all of the methods that you must implement inside the class. You may also implement additional helper methods inside the class as you see fit.

The `UseCases` class is provided to illustrate a few use cases on how the music catalog may be used. It is basically a set of catalog simulations that can be run once you have implemented your music catalog and it helps to model and demonstrate how the `MusicCatalog` class will be used and should behave if it were utilized on a streaming service's platform. It is not a replacement for unit testing which are implemented in the `UnitTests` class; however, you may find it insightful to have another program that uses the music catalog.

The `UnitTests` class comprehensively tests both the `ArrayList` and `LinkedList` implementations. In order to receive full credit for your implementations, your program must pass all tests defined in the provided `UnitTests` class.

3 Requirements

You must implement the following methods in both the `ArrayList` class and the `LinkedList` class using following function signatures:

```
public void add(Song song);
public Song remove();
public Song remove(Song song);
public void clear();
public boolean isEmpty();
public int count();
public Song get(int i);
public boolean contains(Song song);
```

3.1 Classes

This section defines specific requirements for each of the List implementation classes.

3.1.1 ArrayList

For allocations and reallocations of the array list, you must follow these instructions:

- The initial size of the array inside the array list is 2 elements.
- When growing the list, reallocation must follow a doubling strategy. In other words, when reallocating the array, allocate twice the space that was allocated for the previous array.
- The list will never shrink except when “cleared”.
- On `clear`, reallocate the array to its initial size.

3.1.2 LinkedList

Your linked list implementation may only use a `head` pointer and may NOT include a `tail` pointer. We will define linked lists in a future homework that use both `head` and `tail`, but one of the goals of this assignment is to force you to write the `while` loop associated with linked list iteration repeatedly. You must use the `while` loop in your linked list implementation any time that you iterate over the list. In other words, you may not use a `for` loop inside the `LinkedList` class. If a `for` loop is used, you will receive no credit for the implementation of that function.

3.2 Functions

This section gives a general explanation of the requirements for each function. More information may be available in the function headers and in how the functions are unit tested.

void add(Song song)

`add` appends a new element to the end of the music catalog list. For the array-list, this may force a reallocation of the array if there is not enough space which should use a doubling strategy to allocate new space. For the linked-list which has no tail, this will require iteration over the list to find the last element to which the new item is appended. In both cases, you should note that the lists maintain a set of `CatalogItems` rather than raw songs, so you should allocate a new `CatalogItem` and insert the song into that `CatalogItem`. For the array-list, you can ignore the `CatalogItem`’s `next` property which is only used when the `CatalogItem` is used in the linked-list based catalog implementation.

Song remove()

`remove` removes the `Song` at the head of the list (or in the 0th index if the list is array based) if the list is non-empty and returns that song. If the list is empty, `remove` returns `null`. For an array-list, all subsequent elements are shifted down to the preceding index upon removal. `remove` does NOT shrink an already allocated array.

Song remove(Song song)

`remove(song)` removes the first instance of a specified song from the list if one exists and returns that song, and returns `null` if the song does not exist in the list. If a song is removed from an array-list, all subsequent elements are shifted down to the preceding index upon removal. `remove` does NOT shrink an already allocated array. Note that you should probably implement this method after implementing the `contains` method since it is a slightly more complex variant of search.

void clear()

`clear` empties a music catalog of all songs and returns it to its initial state. For an array-list, the array should be reset to its initial allocated length.

boolean isEmpty()

`isEmpty` returns a truth value indicating whether the catalog is empty or not.

int count()

`count` returns the number of elements stored in the catalog.

Song get(int i)

`get` returns the song at the specified index if that song exists in the list; otherwise, `get` returns `null`. If the parameter is invalid, `get` also returns `null`. `get` should not modify the list in any way.

boolean contains(Song song)

`contains` returns true if the specified song exists in the catalog; otherwise, `contains` returns false.

4 Analysis

Along with your code, you must submit an analysis of each of your method implementations in the specified .pdf file.

In your analysis you must analyze the efficiency of each of your method implementations for both the array list and linked list implementations. There are eight methods:

```
public void add(Song song);
public Song remove();
public Song remove(Song song);
public void clear();
public boolean isEmpty();
public int count();
public Song get(int i);
public boolean contains(Song song);
```

You must therefore provide sixteen total analyses, an analysis of each method for the linked list and an analysis of each method for the array list.

Recall that an analysis requires three steps: 1) formulate a hypothesis in terms of Big O, 2) define any analysis variables, generically defined as n where n is the size of the input, 3) defend and justify your analysis using either a diagrammatic, mathematical, or pseudo-code approach. Again, as in your earlier work, your analysis must be “clear, well-reasoned, and well-supported”.